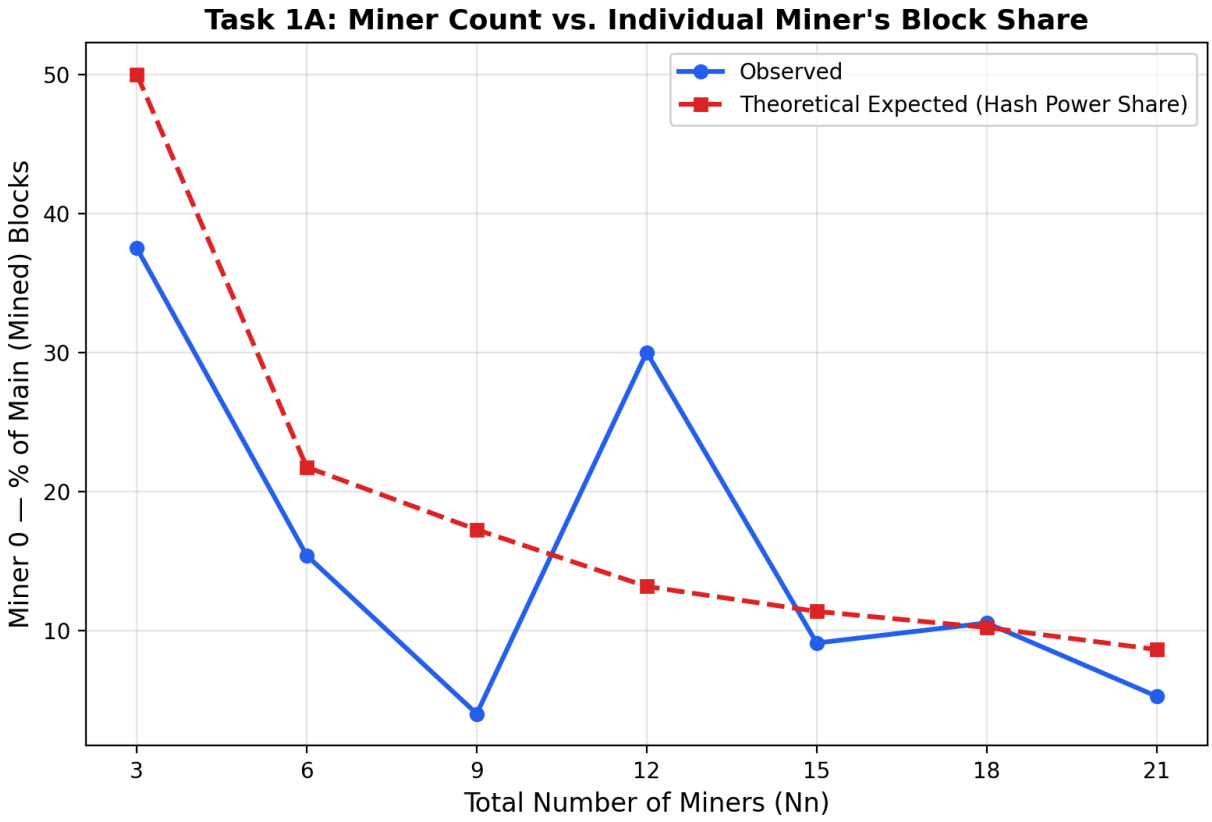
CSE729 — Blockchain and Distributed Ledger Project 1 (Bitcoin Simulation Assignment)

Github Repository - https://github.com/addshakib80/BlockSim_ShakibHossain

Task 1.

A.



For this experiment I kept Miner 0's hash power fixed at 50 while increasing the total number of miners in the network (Nn = 3, 6, 9, 12, 15, 18, 21). Each new miner added was given a varying hash power (some at 10 or 20 or 30 or 70), so the total network hash power kept growing at a different pace each time rather than a fixed amount.

As shown in the graph, Miner 0's percentage of mined blocks generally decreases as the number of miners increases, from 37.5% at Nn = 3 to around 5% at Nn = 21. The decrease is not perfectly smooth because mining is probabilistic and each simulation produces a limited number of blocks. However, the overall trend is clear. Since Miner 0's hash power remains fixed while the total network hash power increases, its share of the network becomes smaller, resulting in a lower percentage of mined blocks.

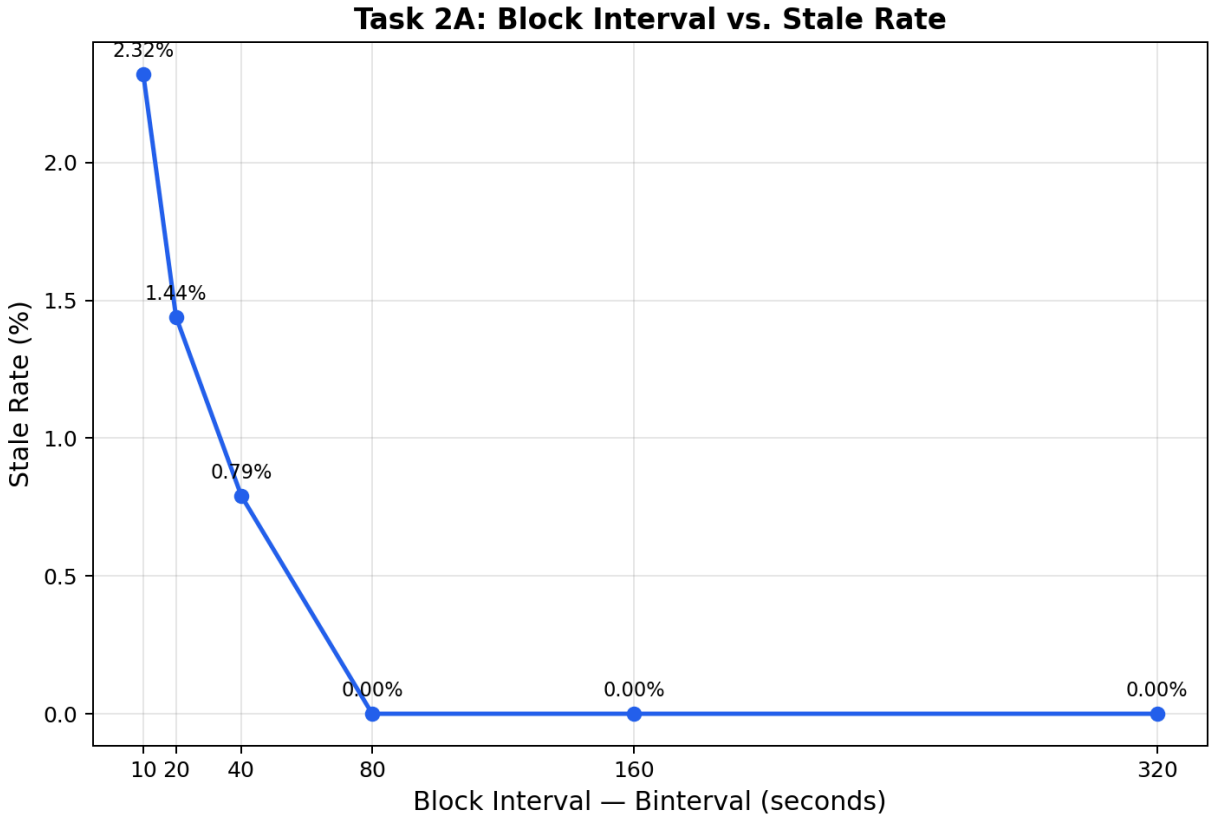
B.

If the total number of miners remains the same, Miner 0's share of the total hash power also remains approximately the same. Therefore, its probability of mining blocks should remain relatively stable over time. Since the block reward also remains unchanged, the miner's profit should increase as it mines more blocks.

For example, in my results, 6 blocks generated 76.49 ETH, while 1 block generated approximately 12.75 ETH. This shows that the profit is mainly determined by the number of blocks successfully mined when the reward per block remains constant. Because mining is probabilistic, some variation is expected between runs.

Task 2.

A.



For this experiment I changed Binterval to 10, 20, 40, 80, 160, and 320 seconds and recorded the Stale Rate from the SimOutput tab each time. The graph shows a clear downward trend. Stale rate drops from 2.32% at Binterval=10 down to 0% once Binterval reaches 80, and it stays at 0% for 160 and 320 as well.

B.
As the block interval increases, the stale rate generally decreases. This is because the block propagation delay remains fixed at approximately 0.42 seconds, regardless of the Binterval value. When the block interval is small, this propagation delay represents a larger portion of the time between blocks, increasing the chance that two miners find blocks around the same time before the first block has fully propagated through the network. This can create competing blocks, with the losing block becoming stale. As the block interval becomes larger, there is more time for a newly mined block to propagate before another block is found, reducing the probability of forks and stale blocks. This is also one reason why Bitcoin uses a relatively long block interval, helping keep the stale-block rate low despite network propagation delays.

Task 3.

A.
I added a variable called useFIFO in InputsConfig.py that can be set to True or False. In Transaction.py, inside the function that picks which transactions go into a block, I added an if/else: if useFIFO is True the transaction pool is left in its original order (FIFO), and if it's False the pool gets sorted by fee, highest first, same as the original code. I also print which ordering mode is active along with the fee values of the first several transactions in the block, so the terminal output actually shows the difference. With fee-priority the printed fees come out in descending order, and with FIFO they come out unsorted.

```
''' Transaction Ordering Parameter '''
        useFIFO = False  # False = fee-priority ordering
(default),
#True = FIFO ordering
```

```
        if p.useFIFO:
            print("Transaction Ordering: FIFO")
        else:
            print("Transaction Ordering: Fee-Priority")
            pool = sorted(pool, key=lambda x: x.fee,
reverse=True)
        print("First 10 tx fees in selected order:",
[round(tx.fee, 6) for tx in pool[:10]])
```

InputsConfig.py | Transaction.py | InputsConfig.pyc

Models > Transaction.py > LightTransaction > execute_transactions

```
42  class LightTransaction():
70      def execute_transactions():
78          # ////////////////////////////////////////////////////////////////////////
79          if p.useFIFO:
80              print("Transaction Ordering: FIFO")# pool is left as-is – it already reflects arrival order
81          else:
82              print("Transaction Ordering: Fee-Priority")
83              pool = sorted(pool, key=lambda x: x.fee, reverse=True)
84          print("First 10 tx fees in selected order:", [round(tx.fee, 6) for tx in pool[:10]])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS D:\BlockSim-master> python Main.py
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000463, 0.000422, 0.000419, 0.000405, 0.000365, 0.000354, 0.00035, 0.000346, 0.000346, 0.000345]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000558, 0.000553, 0.000478, 0.000404, 0.000395, 0.000392, 0.000361, 0.00036, 0.000345, 0.000341]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000571, 0.000514, 0.000468, 0.000418, 0.000405, 0.000405, 0.000396, 0.000386, 0.000378, 0.00037]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000482, 0.00047, 0.000378, 0.000371, 0.000366, 0.000349, 0.000345, 0.000343, 0.000341, 0.000341]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000503, 0.000464, 0.000427, 0.000424, 0.000421, 0.000396, 0.00038, 0.000375, 0.000357, 0.000351]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000535, 0.000516, 0.000513, 0.000507, 0.000477, 0.000453, 0.000409, 0.000392, 0.000381, 0.000381]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000498, 0.000478, 0.000438, 0.000418, 0.000379, 0.000364, 0.00036, 0.000354, 0.00035, 0.000337]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000485, 0.000417, 0.0004, 0.000385, 0.00038, 0.00038, 0.000378, 0.000376, 0.000375, 0.000364]
Transaction Ordering: Fee-Priority
First 10 tx fees in selected order: [0.000606, 0.000472, 0.000413, 0.000405, 0.000394, 0.000364, 0.000362, 0.000353, 0.00035, 0.000346]
```


InputsConfig.py | Transaction.py

Models > Transaction.py > LightTransaction > execute_transactions

```
42  class LightTransaction():
70      def execute_transactions():
78          # ////////////////////////////////////////////////////////////////////////
79          if p.useFIFO:
80              print("Transaction Ordering: FIFO")# pool is left as-is – it already reflects arrival order
81          else:
82              print("Transaction Ordering: Fee-Priority")
83              pool = sorted(pool, key=lambda x: x.fee, reverse=True)
84          print("First 10 tx fees in selected order:", [round(tx.fee, 6) for tx in pool[:10]])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\BlockSim-master> python Main.py
Transaction Ordering: FIFO
First 10 tx fees in selected order: [2.6e-05, 6e-06, 0.000224, 1.7e-05, 6.6e-05, 5.2e-05, 0.000205, 9.9e-05, 3.3e-05, 8e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [0.000108, 4.2e-05, 3.5e-05, 4e-06, 0.0, 6.9e-05, 3.7e-05, 2.1e-05, 6.5e-05, 1e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [8.3e-05, 2e-05, 0.000204, 7.1e-05, 3e-06, 1.3e-05, 0.000245, 2.3e-05, 2e-06, 9.5e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [2.3e-05, 8.5e-05, 0.000119, 1.2e-05, 4e-06, 1.1e-05, 1.9e-05, 3.3e-05, 6.4e-05, 6e-06]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [8e-06, 0.000118, 0.000147, 1.8e-05, 0.000196, 0.000211, 7.5e-05, 1e-06, 3.3e-05, 3e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [0.000165, 9.4e-05, 7.2e-05, 6e-06, 1.3e-05, 9e-05, 0.000103, 1.2e-05, 2e-05, 1.6e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [4.7e-05, 8e-06, 0.000199, 0.000121, 1.9e-05, 1.1e-05, 1.3e-05, 6e-05, 5e-06, 4e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [7.4e-05, 4e-05, 8.1e-05, 2.8e-05, 6.4e-05, 4.5e-05, 5.7e-05, 8.7e-05, 0.0, 1.6e-05]
Transaction Ordering: FIFO
First 10 tx fees in selected order: [0.000156, 5.9e-05, 1.5e-05, 7.5e-05, 3e-05, 3.4e-05, 1e-05, 1.8e-05, 2.1e-05, 3.4e-05]
Transaction Ordering: FIFO
```

B.

FIFO has an advantage in terms of fairness because transactions are processed based on when they arrive, regardless of how much fee they pay. This means a user cannot move ahead of an earlier transaction simply by offering a higher fee. It also makes the transaction waiting order more predictable.
However, FIFO can be less beneficial for miners because they cannot choose higher-fee transactions first. With fee-priority ordering, miners can increase their fee revenue by selecting transactions that offer higher fees. Therefore, FIFO is fairer from the user's perspective, while fee-priority ordering provides a stronger incentive for miners to maximize their earnings.

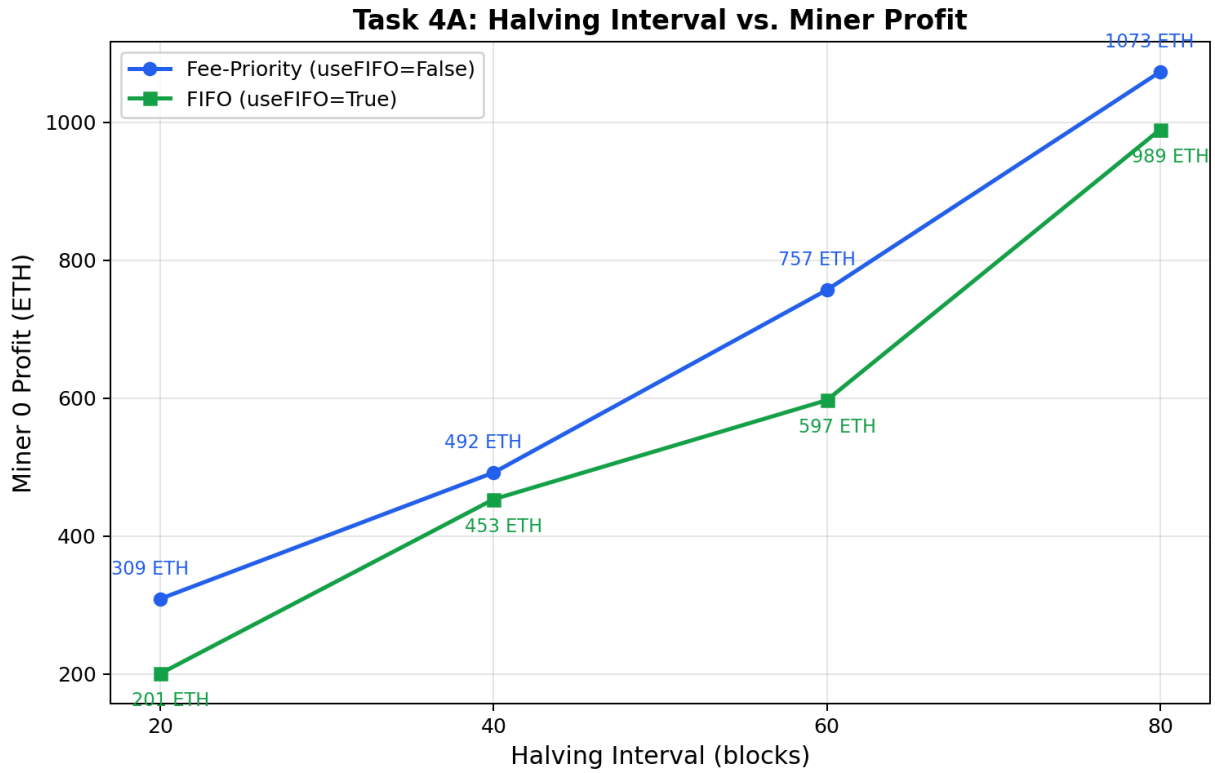
Task 4.

A.
I added a halvingInterval variable and modified Incentives.py so that the block reward is reduced by half after every halvingInterval blocks. The reward is calculated as:

```
def block_reward(depth):
        halvings = depth // p.halvingInterval
        return p.Breward / (2 ** halvings)
```

I tested four halving intervals: 20, 40, 60, and 80 blocks. I set Binterval to 10 seconds so that enough blocks would be mined within the simulation time for the halving mechanism to occur multiple times in all four cases.
Using fee-priority ordering, Miner 0's profit was 309.20 ETH, 492.14 ETH, 756.93 ETH, and 1073.15 ETH for halving intervals of 20, 40, 60, and 80 blocks, respectively.

The results show that profit increases as the halving interval increases. With a shorter interval, the reward is reduced more frequently and becomes very small earlier in the simulation. A longer interval keeps the reward higher for a larger portion of the simulation, resulting in higher total profit. The results also show that fee-priority ordering produced higher profit than FIFO for all the tested intervals.



B.

As the number of halvings increases, the block reward is reduced by half each time. According to the implemented formula, after n halvings the reward becomes:
Breward / 2^n
Therefore, repeated halvings gradually reduce the block subsidy toward zero. Once the block reward reaches zero, miners can no longer rely on newly created coins and would need to earn revenue mainly from transaction fees.

This relates to the transaction ordering experiment in Task 3. Fee-priority ordering allows miners to select higher-fee transactions and therefore provides greater potential fee revenue, while FIFO processes transactions based on arrival order regardless of their fees. In my simulation, fee-priority ordering also resulted in higher Miner 0 profit than FIFO. Therefore, as the block subsidy becomes smaller, transaction fees and fee-priority transaction selection can become increasingly important for maintaining miner incentives.